

A taxonomy of the security controls in Ohtml

Ohtml/1.9.3 · prepared for the OWASP community · every row below is derived from the shipped package, not from its documentation

What this document is

Ohtml is a JSON-to-DOM compiler and a layered hardening engine for single-file HTML applications. It has two distinct halves and they defend different things: a **library** that makes certain outcomes structurally impossible, and an **engine** that applies platform controls to an existing document. This taxonomy separates them, because a reader who conflates the two will over-trust one and under-apply the other.

The layer names, statuses and descriptions were read out of `0html-layers.cjs` by running it. The guarantees and residual risks were taken from `THREAT-MODEL.md`, which the package ships and its own suite traces to. Where this document says a control is not covered, that is a claim from the package, not an omission in it.

1 · Platform controls — the eight layers

Applied to a document that already exists. Each is independent, each states its own precondition, and the engine refuses rather than half-applies when a precondition is unmet.

ID	Layer	Kind	What it does	What it stops
L0	survey & baseline	read-only	Reports CSP posture, innerHTML sinks bucketed by trust, inline handlers, and which layers apply. Changes nothing.	—
L1	CSP baseline directives	additive	Adds the safe missing directives: frame-ancestors, base-uri, object-src, form-action. Merged into any existing policy.	Clickjacking, base-tag hijack, plugin execution, form exfiltration
L2	drop 'unsafe-inline' (hash-pin)	structural	Hash-pins every inline script and removes unsafe-inline; converts inline on* handlers to addEventListener.	Injected script or handler cannot execute even if a sink regresses
L3	Trusted Types (report-only)	diagnostic	Adds require-trusted-types-for in Report-Only and registers the policy. Non-breaking; surfaces which sinks are not policy-routed.	Nothing yet — it tells you what L4 would break
L4	Trusted Types (enforce)	structural	Routes sinks through the policy and flips to enforce.	DOM-XSS at the platform layer, including future unescaped-sink regressions
L5	connect-src allowlist	scoping	Replaces connect-src * with the app's actual endpoints.	Exfiltration through the broadest hole
L6	secret isolation	advisory	Pattern and advice for keys in storage: Web-Crypto wrap, session-only, or backend.	Blast radius once a rogue script is already running
L7	JSON-compiler adoption	structural	Renders some or all UI through the allowlist compiler.	Everything in Part 2, on those paths only
L8	style-src hardening	structural	Hash-pins inline style blocks and drops unsafe-inline from style-src. Blocks if inline style= attributes are present.	CSS injection and CSS-based exfiltration

L2 and L8 are the two that change what the browser will execute. L3 exists so that L4 can be adopted without discovering its breakage in production, and skipping it is the common failure.

2 · Library guarantees — what a spec cannot do

These hold for UI rendered through the compiler. They are not advice or defaults: they are properties the input language cannot express. Each maps to STRIDE and to executable tests.

ID	Guarantee	STRIDE
G1	No spec can cause script execution.	E, S
G2	No spec can inject an active element — script, iframe, object, embed.	E
G3	No spec can attach an event handler that is not already registered in the caller's allowlist.	E
G4	No spec can pollute Object.prototype.	T
G5	No spec can produce a javascript: or data: URL sink.	I
G6	No spec can exhaust the stack, or the heap unboundedly, via structure.	D
G7	Raw HTML renders only from a caller-allowlisted generator.	E
G8	All of G1–G7 hold under Trusted Types enforce mode, not only without it.	E
G9	A pinned tool is tamper-evident: bundle drift or a stale CSP hash is detected.	T, R

G9 is the operational one and the least often adopted. Without it the other eight describe a bundle that may not be the bundle you are running.

3 · What is deliberately not covered

The package states these itself. They are reproduced here unchanged because a taxonomy that lists only what a tool does is an advertisement. Each is a control someone else must own.

ID	Not covered	Why it is someone else's control
N1	Sanitising inside trusted HTML	<i>trust() content is the generator's responsibility, not the compiler's</i>
N2	Restricting outbound hosts	<i>a scheme-valid https://evil.example passes URL validation — that is L5's job, not the library's</i>
N3	Sandboxing CSS	<i>style is accepted as a string; L8 addresses the policy half, nothing addresses the content half</i>
N4	Constraining handler behaviour	<i>a spec cannot add a handler; what an allowlisted handler then does is the application's problem</i>
N5	A compromised bundle at runtime	<i>verify is build-time. Integrity of the served bytes rests on the host</i>
N6	Replacing platform hardening	<i>the library guarantees are not a substitute for CSP — pair them</i>
N7	L4 as a sanitiser	<i>the default Trusted Types policy is a passthrough. It buys an audit choke point, not sanitisation</i>
N8	style-src parity with script-src	<i>script-src is fully hardened; style-src is partial</i>
N9	connect-src as exfiltration cover	<i>it is not all exfiltration — images, navigation and DNS remain</i>
N10	Parser equivalence	<i>the converter uses a real scanner as of 1.9.x; earlier versions did not</i>
N11	Marker trust	<i>the engine trusts its own markers, so a file of untrusted provenance can lie about what is applied</i>
N12	Conformance as a floor	<i>bare conformance verifies a claim, not a floor, unless gated with --require</i>

4 · Reading this as an OWASP practitioner

The mapping below is offered, not asserted by the package. Ohtml carries one CWE reference in its documentation, so treating these as authoritative would be over-claiming; they are the categories a reviewer would reach for.

OWASP Top 10 (2021)	Controls	Caveat
A03:2021 Injection	L2, L4, L7, G1–G3, G5	DOM-XSS specifically. G7 is the boundary: raw HTML from an allowlisted generator only.
A05:2021 Security Misconfiguration	L1, L5, L8	These add directives that are absent by default in most single-file apps.
A08:2021 Software & Data Integrity	G9, L2 hashes	Build-time tamper evidence. N5 is the limit: runtime integrity is the host's.
A01:2021 Broken Access Control	—	Out of scope. Nothing here authorises anything.
A02:2021 Cryptographic Failures	L6 (advisory only)	L6 is a pattern, not an implementation. Do not record it as adopted.

5 · How to check any of this yourself

Command	What it establishes, and what it does not
<code>node 0html-layers.cjs layers <file></code>	the layer menu for your document, with each layer's precondition and whether it is blocked
<code>node 0html-cli.cjs verify</code>	bundle, pinned toolchain and SBOM match the lock. Build-time only — see N5
<code>node 0html-layers.cjs conformance --require</code>	the gated form. The bare form returns 0 on a document with no CSP at all — see N12
<code>node 0html-layers.cjs audit-sri</code>	flags external subresources with no integrity attribute
<code>node 0mutate.cjs</code>	breaks each guard in turn and checks the suite notices. 21 guards, all caught

The last one is the one worth running twice. A suite that passes tells you it ran; a mutation harness tells you it would have failed. Every guarantee in Part 2 is traced to a named test file in THREAT-MODEL.md section 8.

A closing caution

Two controls in this package were found dead in a browser while every static check reported green. Nothing in Parts 1 or 2 is confirmed by reading a report: a Content Security Policy is only enforced by a browser that received it, and a toolchain cannot observe enforcement. If you adopt these layers, load the deployed page and read the console for refusals. That step is the one this taxonomy cannot do for you.